

The PM's Guide to Agent Distribution: MC Servers, CLIs, and AGENTS.md

Stripe, Cloudflare, Linear, and 7 others already shipped. Audit template, PRD, and Clerk Code sprint included.

AAKASH GUPTA

MAR 07, 2026 • PAID



93



2



7

Share

Your product's next million users won't have eyes.

They won't visit your website, sit through your onboarding, or read your changelog. They'll be AI agents, autonomous software that discovers, evaluates, and integrates tools *programmatically*.

Quote me on this:

If your product cannot be parsed, authenticated, and executed by an agent, you are invisible in the fastest-growing software channel.

It sounds futuristic. It isn't. Stripe, Cloudflare, Shopify, GitHub, Asana, Zapier, and others have already shipped agent-facing surfaces.

Andrej Karpathy put it bluntly last week: "It's 2026. Build. For. Agents."



Andrej Karpathy ✓
@karpathy



CLIs are super exciting precisely because they are a "legacy" technology which means AI agents can natively and easily use them, combine them and interact with them via the entire terminal toolkit.

E.g ask your Claude/Codex agent to install this new Polymarket CLI and ask for any arbitrary dashboards or interfaces or logic. The agents will build it for you. Install the Github CLI too and you can ask them to navigate the repo, see issues, PRs, discussions, even the code itself.

Example: Claude built this terminal dashboard in ~3 minutes, of the highest volume polymarkets and the 24hr change. Or you can make it a web app or whatever you want. Even more powerful when you use it as a module of bigger pipelines.

If you have any kind of product or service think: can agents access and use them?

- are your legacy docs (for humans) at least exportable in markdown?
- have you written Skills for your product?
- can your product/service be usable via CLI? Or MCP?
- ...

It's 2026. Build. For. Agents.

```
~/bin/polymarket-dashBoard
```

VRMARKET Feb 24 10:15

Event	Total	24h	#1	Price	24h	#2	Price	24h	#3	Price	24h	#4	Price	24h	#5	Price	24h
Democratic Presidential Nominee 2.	\$787.6M	\$0.3M	Gavin Newsom	27%	+0.4	Alexandria Oc.	9%	+0.2	Kamala Harris	6%	+0.1	Jon Ossoff	4%	+0.1	Josh Shapiro	4%	+0.1
Who will Trump nominate as Fed Ch.	\$512.1M	\$1.2M	Kevin Marsh	94%	+0.1	Judy Shelton	4%	+0.3	No one nomina.	<1%	-	Christopher M.	<1%	-	Stephen Miran	<1%	-
Presidential Election Winner 2028	\$525.4M	\$5.8M	JD Vance	23%	+0.1	Gavin Newsom	17%	+0.5	Marco Rubio	7%	+0.1	Alexandria Oc.	4%	+0.5	Kamala Harris	4%	+0.5
Republican Presidential Nominee 2.	\$125M	\$1.5M	J.D. Vance	42%	+1.2	Marco Rubio	14%	+0.9	Donald Trump	2%	-	Donald Trump	2%	+0.1	Ron DeSantis	2%	+0.1
2026 NBA Champion	\$277.6M	\$2.1M	Oklahoma City	35%	-	San Antonio S.	12%	+3.5	Denver Nuggets	12%	-	Detroit Pist.	9%	+0.4	Cleveland Cav.	9%	+0.4
English Premier League Winner	\$26.7M	\$12M	Arsenal	63%	-	Man City	37%	-	Man United	1%	+0.1	Aston Villa	<1%	+0.1	Liverpool	<1%	+0.1
UEFA Champions League Winner	\$24.2M	\$1.9M	Arsenal	19%	-	Bayern Munich	18%	+1	Barcelona	11%	-	Man City	10%	-	PSG	10%	-
2026 FIFA World Cup Winner	\$185.8M	\$5.2M	Spain	15%	+0.1	England	11%	+0.1	Argentina	11%	+0.5	France	11%	+0.1	Brazil	11%	+0.1
La Liga Winner	\$168M	\$1.1M	Barcelona	48%	+0.5	Real Madrid	37%	-	Villarreal	<1%	+0.3	Betis	<1%	-	Atletico Madr.	<1%	-
NBA MVP	\$19.1M	\$907.4K	Shai Gilgeous	54%	+0.5	Nikola Jokic	28%	+2.9	Cade Cunningham	12%	+1.6	Victor Wemban.	4%	+1.2	Luka Doncic	4%	+1.2

▲ up ▼ down (x0.1%) Prices = probability in cents

The post named something product teams have been slow to act on: **the distribution channel is shifting from human interfaces to agent interfaces.**

Today's Post

This is the most practical guide I could write on how to own this shift as a PM:

1. The 5 Distribution Channels - and Why #5 Changes Everything
2. The Agent-Accessible Product Stack
3. The PM's Role in Agent Distribution
4. The Strategy Meeting, Pitch + PRD
5. Analyzing 10 the top MCP Servers
6. Building It - Claude Code Sprint
7. Where Teams Go Wrong

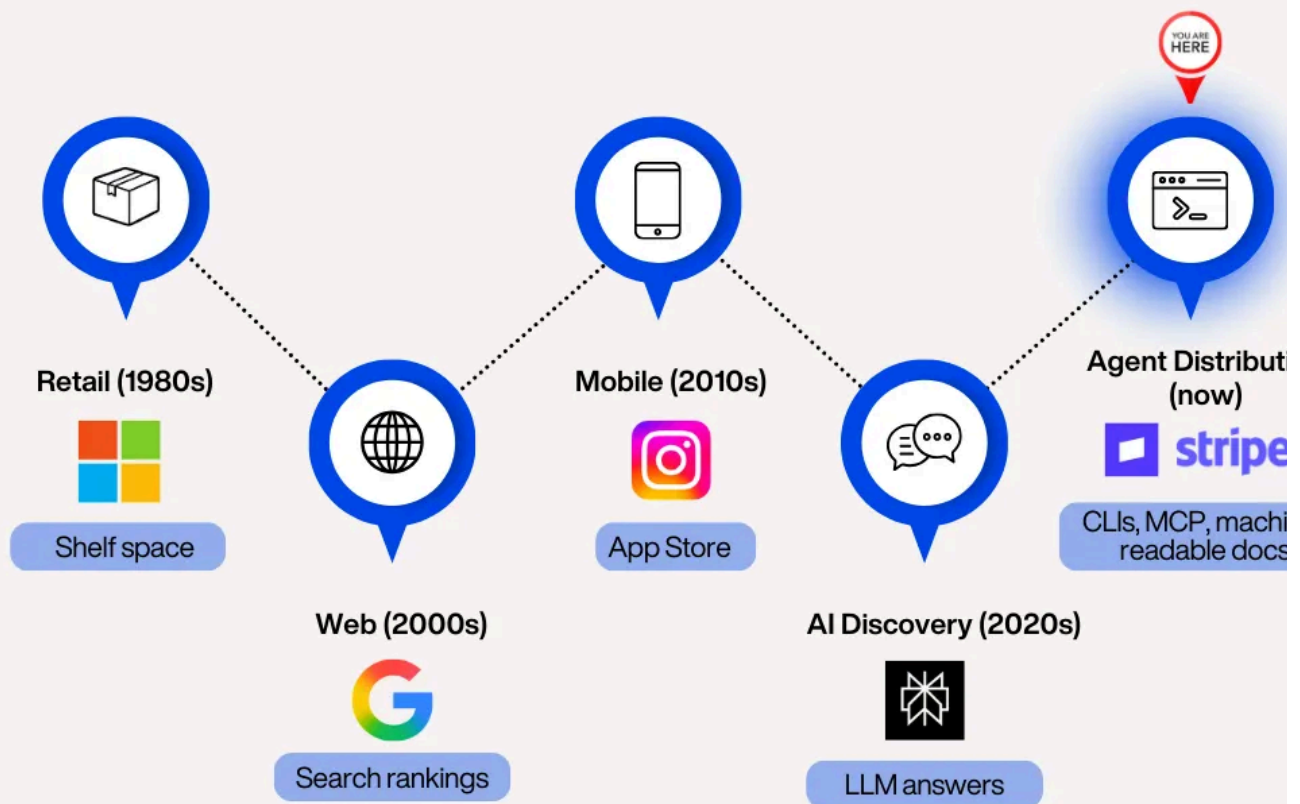
Sections 1 and 2 are free. Sections 3-7 are for paid subscribers.

1. The 5 Distribution Channels - and Why #5 Changes Everything

Every generation of software has a dominant distribution channel. Every time the channel shifts, the companies that build for the new interface first win the decade.

The 5 Distribution Channels

Aakash G

**97M**

MCP downloads

10,000+

Servers

40%

enterprise apps embedding agents by end of 2026.

Channel 1: Retail (1980s-90s)

Software came in boxes. You drove to CompUSA. Distribution meant shelf space. Microsoft won by getting Windows pre-installed on PCs.

Channel 2: Web (2000s)

Google changed everything. Distribution shifted to search rankings, landing pages and self-serve signups. Salesforce killed on-premise with a URL. I wrote about how this shaped [activation](#) and [onboarding](#) as core growth levers, and why [PLG](#) became the dominant go-to-market.

Channel 3: Mobile (2010s)

The App Store created a new discovery surface. Companies that designed for smartphones from day one (Instagram, Uber, WhatsApp) ate companies that tried to port to desktop. I covered how [growth loops](#) compound differently on mobile.

Channel 4: AI Discovery (2020s)

LLMs started answering questions instead of linking to websites. If ChatGPT describes your competitor instead of you, you lose the click entirely. I wrote about this shift in my [AEO/GEO guide](#), the new discipline of making your product visible to answer engines.

Channel 5: Agent Distribution (Now)

This is the one Karpathy is talking about. Agents don't discover products through search results or app stores. *They discover them through CLIs, MCP servers, and machine-readable documentation.* They don't onboard. They don't browse. They connect, authenticate, execute, and move on.

Each channel shift followed the same pattern: the companies that built for the new interface first captured outsized market share. The ones that tried to retrofit lost ground they never recovered.

This shift **appears to be happening faster than most interface transitions.** Major platforms, model providers, and infrastructure vendors are aligning *unusually quickly*.

The Numbers

MCP, the [Model Context Protocol](#), went from zero to [97 million monthly SDK downloads](#) across Python and TypeScript in its first year.

[10,000+ active MCP servers](#). OpenAI, Google DeepMind, Microsoft, and Cloudflare adopted it. In December 2025, Anthropic donated MCP to the Linux Foundation [Agentic AI Foundation](#) because the standard had already emerged as the early leader. OpenAI and Block joined as co-founders. Google, Microsoft, AWS, Cloudflare, and Bloomberg as supporting members.

Gartner [projects](#) that 40% of enterprise applications will embed task-specific AI by end of 2026, up from less than 5% in 2025. That's one of the steepest adoption curves in enterprise history.

So the channel is real. The adoption is real. The question for PMs is: *what do you actually build?*

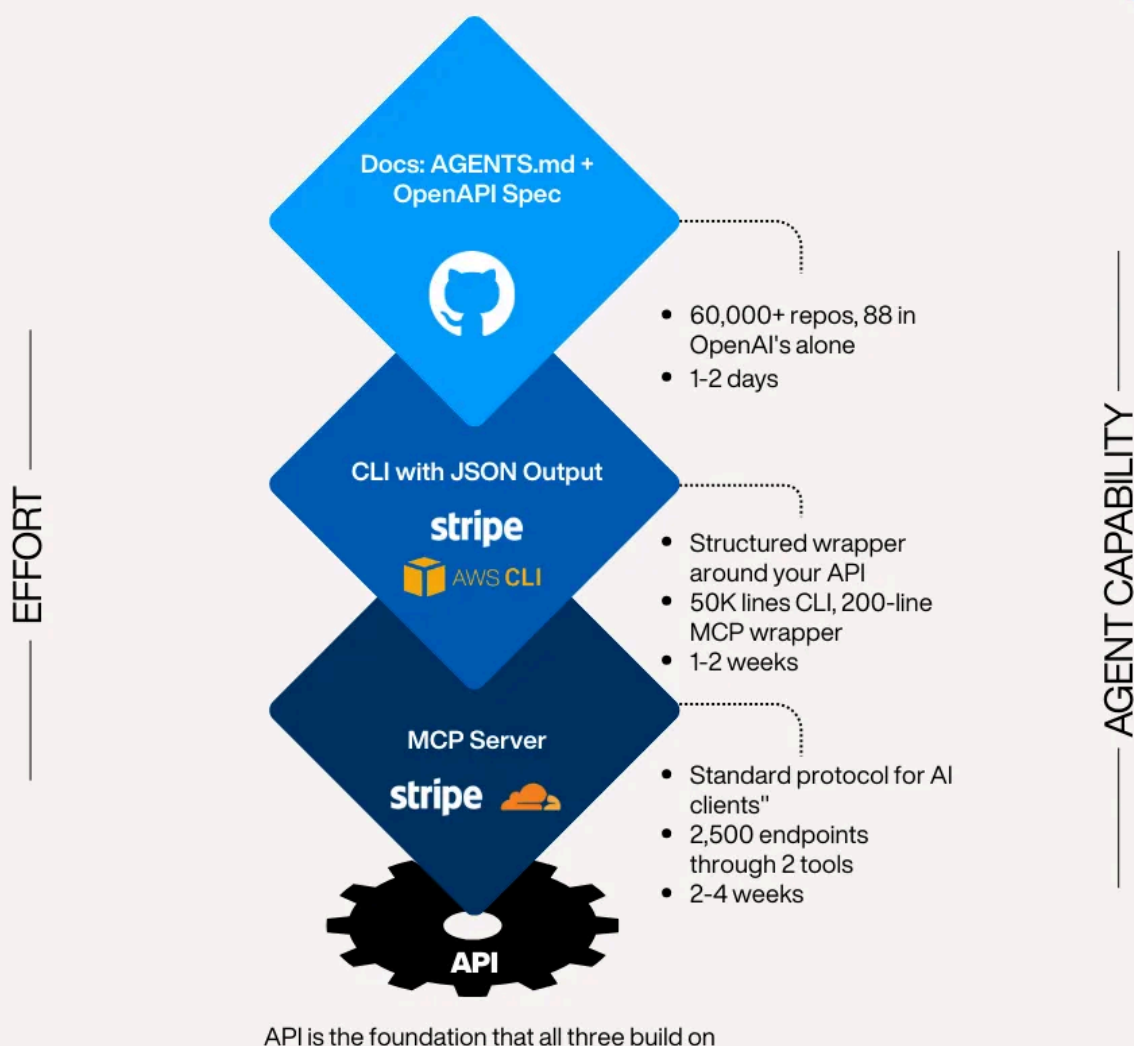
2. The Agent-Accessible Product Stack - What to Build First

Your API is always the foundation. MCP servers call it under the hood. CLIs call it under the hood. If your API is a mess, everything above it will be too. If your auth is broken, neither MCP nor a CLI will fix it.

On top of your API, you build three surfaces. Most companies end up with all three. They layer, they don't replace each other, and the order you build them matters.

The Agent Access Stack

Aakash C



10 companies launched production MCP servers in one day, May 2025

Layer 1: Documentation

[AGENTS.md](#) is the emerging standard for telling agents how to work with your codebase and product. It emerged from collaborative efforts across OpenAI Codex, Google's Jules, Cursor, GitHub Copilot, and Amp.

Think of it as a README, but written for machines instead of humans. OpenAI donated it to the [Agentic AI Foundation](#) alongside MCP.

GitHub analyzed 2,500+ AGENTS.md files and found a clear split between ones that work and ones that don't. The effective ones put executable commands early, structured code instead of prose, set clear boundaries on what agents should never do, and specified exact framework versions. [60,000+ projects](#) have already adopted the format. OpenAI's own repo has 88 AGENTS.md files, one per package.

The companion to AGENTS.md is an **OpenAPI spec**. If your [API docs](#) are scattered across Notion pages and README files, agents can't parse them. An OpenAPI 3 spec is a machine-readable contract: every endpoint, every parameter type, every code in one structured file. Tools like Speakeasy and Gravitee can auto-convert a OpenAPI spec into an MCP server. No spec, no conversion.

Vercel is pushing this further with **Agent Skills**: packaged instruction folders (SKILL.md files) that give agents specialized expertise. Stripe, Cloudflare, Supabase, Sentry, Expo, and Hugging Face all publish official [agent skills](#). They work across Claude Code, GitHub Copilot, Cursor, Gemini CLI, and others. It's another documentation surface that's quietly becoming a distribution channel.

That's where every team should start.

Layer 2: CLI

Karpathy's point about CLIs being "legacy" technology is precise. Legacy means battle-tested, standardized, universally parseable.

The entire Unix philosophy was *accidentally designed for [AI agents](#)* decades before it existed: small composable commands, JSON output piped between tools, environment variable auth, structured exit codes.

A **quick clarification** on how CLIs relate to APIs, since this trips up a lot of PMs:

- Your API is the **engine**. It's the raw programmatic interface.
- A CLI is a **structured wrapper** around your API that makes it composable: piping output to grep, chaining commands with jq, using environment variable authentication.

- An MCP server is **another wrapper**, this one designed specifically for AI clients to discover and call tools through a standard protocol.

They're layers, not alternatives, and each adds a new surface for different types of consumers.

Stripe's CLI lets you pipe JSON responses to jq, chain subscription queries with customer lookups, export data in CSV. That existed years before MCP.

Now agents use it natively. Same story with AWS CLI, Terraform, Docker, GitHub, and Vercel's CLI.

These companies built CLIs for developer productivity and got agent-accessible infrastructure for free.

Layer 3: MCP Server

This is where the biggest investment happens, but it's also where the quality gap between companies is widest. An MCP server exposes your product's capabilities as tools that Claude, ChatGPT, Cursor, Copilot, and thousands of other clients understand through a single [standard protocol](#).

For many developer and B2B tools, an MCP server is quickly becoming *table stakes*. Having a *good* one is the **competitive advantage**.

What Makes a Good MCP Server

The difference between good and bad comes down to a few things most teams get wrong:

1. **Tool descriptions.** This is where most MCP servers fail. If your tool says "manages tasks," an agent doesn't know whether it creates, reads, updates, or deletes. Research on [MCP tool selection](#) shows agents start failing at 30+ tools when descriptions overlap, and virtually guarantee wrong picks at 100+. The Speakeasy team found that reducing Playwright's MCP server from 26 tools dramatically improved agent accuracy. Design tools around outcomes, not

endpoints. One `track_order(email)` that calls three APIs internally beats three separate tools the agent has to chain.

2. **Auth without a browser.** OAuth 2.0 device flows work: show a URL and code in the terminal, user approves in browser, agent gets the token. API keys for server-to-server. *No browser-dependent auth in the critical path.*
3. **Structured errors.** “API_TOKEN is invalid, reconfigure the MCP server” is actionable. “Access denied” is not.
4. **Idempotent endpoints.** Agents retry. They will call the same endpoint multiple times on a timeout. Handle duplicates gracefully.
5. **Clear rate limits.** Return 429 status codes with Retry-After headers. Agents that hit opaque limits will either crash or hammer you exponentially.

You want to **push your team** to get to this level.

Everything below is for paid subscribers.

*The free section tells you **what** this channel is. The paid section gives you the part most teams will actually need help with:*

- ***The 5-question audit** to score agent readiness*
- ***The stakeholder pitch** to win leadership buy-in*
- ***The PRD template** for your first agent-access initiative*
- ***The Claude Code sprint** from zero to working implementation*
- ***10 production teardowns** to copy the right patterns*
- ***The 7 mistakes** that make agents ignore your product*

Most posts on this topic explain MCP. This section shows you how to actually get the initiative approved and shipped.

3. The PM's Role in Agent Distribution

So you're a PM reading this. Maybe your company has a CLI. Maybe it doesn't. Maybe you've heard of MCP. Maybe this is the first time.

Here's where you actually fit:

- If you're at a *developer tools company* with an existing API and CLI...
You're closer than you think. Your CLI already outputs structured data. You already handles authentication. The gap is probably an MCP server and better documentation. *2-4 weeks, 1-2 engineers.*
- If you're at a *B2B SaaS company* with an API but no CLI...
You have a bigger lift but a clear path. Start with AGENTS.md and an OpenAPI spec. Get your API docs in shape. Then scope a CLI or jump straight to MCP for your top 5 use cases.
- If you're at a company where most value is locked behind a GUI with limited API coverage...
You need to start earlier in the stack. The first conversation is about API coverage, not MCP. Agent access is a forcing function for building the [programmatically accessible API](#) you probably should have built anyway.

The PM's job across all three scenarios is the same: frame the opportunity, define what to build first, write the spec, and *own the quality of the agent experience.*

That last part is where you add the most value.

Engineers will build the MCP server. They'll expose the endpoints and handle the auth. But the quality of the [tool descriptions](#)? The scoping of which features to expose first? The decision to start read-only? The choice of which use cases agents will actually care about?

That's [product judgment](#).

Look at how Stripe writes their tool descriptions. An agent reading “review payment, troubleshoot declines, process refunds” immediately knows what the tool does and

when to use it. Compare that to “manages payment operations.” The first one gets picked. The second one gets skipped.

Docker's MCP team put it directly:

“The agent, not the user, is the one calling your tool. Your documentation needs to suit both audiences.”

The PM is the person who understands both.

Your role is to be the voice of the agent user. Just like you advocate for human users, you now advocate for the agent trying to figure out your product at 3am with no [onboarding flow](#), no tooltips, and no customer success manager.

4. From Strategy to Spec: The Meeting, The Pitch, The PRD

You understand the landscape. You know the PM's role. Now you need to actually get alignment, scope the work, and write the spec.

This section gives you the exact playbook to go from “we should think about agent access” to “here's the PRD, here's the timeline, here's who owns what.”:

Step 1 - The Audit

Before anything else, answer five questions about your product:

1. What percentage of core value is accessible through an API?
2. Do you have a CLI? Does it output JSON?
3. Are your API docs machine-readable (OpenAPI spec)?
4. Can a developer authenticate without a browser?
5. Do you have an AGENTS.md?

Most teams score 1-2 out of 5. That's fine. It tells you where to start.

Step 2 - The Strategy Meeting

Your audit score is your conversation starter. You need alignment from eng leads and probably your VP/CPO. I'd keep it to 45-60 minutes.

Start with data and a demo. Show Stripe's MCP docs page. Show Linear's one-command setup. If you can, open Claude Code, connect to one of their MCP servers and show your team what it looks like when an agent uses a product. *Seeing an agent create Linear issues from conversation hits differently than reading about it.* Frame this distribution channel shift, not a feature request.

The next question is which features agents would actually want. Data retrieval, searches, checks, simple CRUD, report generation, and monitoring are the sweet spots. Cross-reference with what already has API coverage. The overlap is your starting scope.

From there, scope a concrete milestone. If you scored 0-1 on the audit: AGENTS plus OpenAPI spec in 2 weeks. If 2-3: CLI or MCP server on a 4-week timeline. I'd frame it as a sentence:

“By [date], an agent should be able to [specific action] through our [interface].”

One more thing: assign a PM as owner, not a tech lead. The PM owns the agent experience. 1-2 engineers, 2-4 weeks.

Step 3 - The Stakeholder Pitch

If you need to sell this upward:

“Agent access is the next distribution channel. Stripe, Linear, Sentry, Asana, Shopify, Cloudflare, GitHub, Notion, Zapier, and Pendo all shipped production MCP servers. We hit 97 million monthly SDK downloads. Gartner projects 40% of enterprise apps will use AI agents by end of 2026. The investment is 1-2 engineers for 4 weeks. The risk of no action is that competitors who ship agent access first become the default tool in every autonomous workflow.”

Step 4 - The PRD Template

I couldn't find a good PRD template for this anywhere, so I built one. You can steal it as-is for your next strategy review, or hand it to Claude Code and have it draft your first version in minutes.

To use this: either paste it into your existing [PRD tool](#), or feed it to Claude Code [PM Operating System](#) with context about your product and it'll fill it out for you

AGENT ACCESS PRD

Problem Statement

[X]% of our product's value is locked behind a GUI. As AI agents become a primary way workflows get assembled, our product is invisible to agent-based discovery. Competitors [name them] have shipped [CLI/MCP/structured docs] and are capturing agent-driven usage.

User (Agent) Persona

The "user" is an AI agent (Claude Code, Cursor, Codex, custom enterprise agents) acting on behalf of a human. The agent needs to: discover capabilities, authenticate programmatically, execute through structured interfaces, return parseable results. The agent's human principal may never directly interact with our product.

Goals: complete a task in one shot. Constraints: can only see machine-readable content. Success: correct result with no confusion from vague descriptions or broken auth.

Success Metrics

- Agent-originated API calls as % of total (target: X% within 6 months)
- Number of MCP client integrations (target: listed in X registries)
- Time-to-first-action for an agent (target: under 5 minutes)
- Error rate on agent requests (target: below X%)

Scope: Phase 1

- AGENTS.md file in primary repo
- OpenAPI 3.0 spec for all public endpoints
- CLI with JSON output for top 5 use cases: [list them]
- OAuth 2.0 device flow for programmatic auth
- MCP server exposing [X] read-only tools with full schema definitions

Tool Definitions (for MCP)

For each tool:

- Name: action-oriented, snake_case (e.g., create_task, get_report)
- Description: what it does + when to use it (1-2 sentences)
- Input schema: JSON schema with types, required/optional, descriptions
- Output schema: what the agent receives
- Annotations: readOnlyHint, destructiveHint, openWorldHint

Phase 2: Write access with approval workflows, agent analytics dashboard

Phase 3: Agent-specific pricing, marketplace listings

Non-Goals:

- Replacing the existing GUI
- Agent pricing in Phase 1
- Full write access on day one

Step 5 - Making Agent Access Part of Every PRD

The template above is for your first agent-access initiative. But every feature PRD write from now on should include an agent access section. Not a separate doc. Part of the feature definition.

Here's my current PRD for [feature name]:

[Paste PRD]

Add an "Agent Access" section: How would an agent discover and use this feature? What MCP tool is needed? Input/output schema? Error states? Agent usage metrics?

Integrate as a subsection. Agent access is part of "done."

If agent access lives in a separate PRD, it becomes someone else's problem. If it's a section in every feature PRD, it becomes culture. This is the single highest-leverage change you can make.

5. What Good Looks Like - 10 Production MCP Servers

Once you have a draft strategy and PRD, the next question is simple: **what does success actually look like in production?**

That's where examples help. Here's what I learned studying 10:

1. [Linear](#) exposes tools for issues, projects, and comments. OAuth 2.1. Setup is command: `claude mcp add --transport http linear-server https://mcp.linear.app/mcp`. An agent can triage bugs, plan sprints, rebalance workloads, create project structures from conversation. Their engineering team described it as “bringing Linear directly into AI tools of cl I wrote about [how Linear grows](#) and agent access is now a core part of that s
2. [Sentry](#) at `https://mcp.sentry.dev/mcp` lets developers query errors from their IDE or Claude. Then they played a meta-game: built MCP server *instrumentation* so you can monitor your own MCP server using Sentry. Build MCP for your product AND the monitoring tools for everyone else's. That's you become infrastructure.
3. [Asana](#) connects to their Work Graph. Project updates, task search, comment deadline management. Enterprise customers control which MCP clients are allowed. Their positioning: “Meet users where they are, enabling agent-driven interoperability.”
4. [Shopify](#) went further than anyone. They built MCP UI, returning *interactive components*, not just data. Agent searches for products, gets back embedded product cards with variant selection, image galleries, and cart flows. They open sourced the spec. Shopify treats MCP as a first-class commerce channel alongside their storefront.
5. [Stripe](#), covered in Section 1, sets the standard for tool descriptions. An agent reads “review payments, troubleshoot declines, process refunds” and immediately knows what the tool does and when to use it. Their Order Intents API lets agents complete autonomous purchases. Visa Agent Cards add spending controls. If they ship a Terraform provider so agents can manage Stripe configuration through infrastructure-as-code, not just API calls.
6. [Cloudflare](#) took the most radical compression approach. Over 2,500 API endpoints across DNS, Workers, R2, and Zero Trust, all routed through two `search()` to find the right API call and `execute()` to run it. The agent wraps JavaScript against a typed OpenAPI schema, Cloudflare runs it in a sandbox worker. The entire interaction fits in roughly 1,000 tokens. If you have a mas

API and you're wondering how to avoid exposing hundreds of MCP tools, st Cloudflare.

7. [GitHub](#) ships one of the most full-featured MCP servers in production. Repo issues, PRs, code review, CI/CD, Projects. What makes it worth studying: dynamic toolset discovery (agents enable tools on demand via an X-MCP-Tools header), server instructions that guide multi-step workflows through system prompts, lockdown mode that restricts what untrusted contributor content can trigger. They consolidated related tools into fewer multi-operation tools with a `meta` parameter, cutting context window usage by 50-90%. And PAT scope filtering automatically hides tools your token can't access, so agents never see what they can't use.
8. [Notion](#) offers both a hosted remote MCP server (one-click OAuth) and a self-hostable open-source version. The design lesson: their v1 returned hierarchical block JSON, which ate context windows. V2 converts everything to Notion-flavored Markdown. Same information, dramatically fewer tokens. If your product stores hierarchical data (docs, wikis, knowledge bases), flatten it for agents.
9. [Zapier](#) turns 30,000 actions across 8,000 apps into MCP tools. One connection gets an agent access to Slack, Gmail, CRMs, analytics, and everything else. Their CTO [joined me on the Product Growth Podcast](#) for a live demo. The scale is impressive but it's also a warning: at 30K+ tools, description quality and smart routing become existential. "It'll work one time then go off the rails" if tool descriptions overlap.
10. **Pendo** brings product analytics into agent workflows. Ask Claude to compare engagement across segments, and Pendo's MCP server returns interactive charts you can toggle between line and bar, export the underlying data, and ask follow-up questions inline. Their CEO [joined the podcast](#) to talk about this approach. If you're in analytics or reporting, Pendo shows what's possible when you return visualizations alongside data instead of raw JSON tables.

The goal here is *not to copy any one MCP server exactly*. It's to **study the decisions** strong teams made: what capabilities they exposed, how they described tools, how

handled auth, and where they kept the surface area narrow instead of trying to expose everything at once.

Across these ten examples, **four architecture patterns** show up again and again.

1. **Outcome-Oriented** servers map tools to workflows, not endpoints. One `track_order(email)` that calls three APIs internally beats three separate endpoints the agent has to chain. Stripe and Linear do this.
2. **Minimalist** servers compress massive APIs into a few meta-tools. Cloudflare routes 2,500 endpoints through two tools. The agent writes JavaScript against a JSON typed schema, runs it in a sandboxed worker, and gets back the result. About 100k tokens instead of over a million if they'd mapped every endpoint.
3. **Platform Play** servers ship MCP alongside companion tools: monitoring, UI components, CLI, Terraform providers. Sentry, Shopify, and GitHub all take this approach.
4. **Knowledge Bridge** servers optimize for how agents consume information. They use efficient formats, interactive responses, smart routing across large tool sets. Notion, Zapier, and Pendo fit here.

Most mature servers blend patterns. Stripe is outcome-oriented AND a platform (MCP + Terraform + Visa Agent Cards). The pattern you start with depends on your API surface and customer base.

The pattern across all ten: **none of these are side projects**. They're maintained by teams, with OAuth or API keys, enterprise controls where needed, and listings on discovery registries. *That's the bar.*

6. Building It: A Claude Code Sprint

If you have [Claude Code](#) access, open it in your project directory and follow along. You're going to go from zero to a working AGENTS.md, CLI, and MCP server in a single sitting.

If you don't have Claude Code, use this section as a spec for what to build with your engineering team. The prompts become ticket descriptions.

Audit and Docs

Start with the audit:

```
> Look at our API and codebase. Audit our product for agent
    accessibility. What's exposed? What's missing?
    Gap analysis ranked by effort and impact.
```

Claude Code scans your codebase, finds your API routes, tells you what's there and what isn't. Use the output to confirm your PRD scope.

Then generate the AGENTS.md:

```
> Create an AGENTS.md for this project. Product capabilities in
    plain language, auth with example commands, top 5 use cases
    with requests and responses, rate limits, and boundaries
    (what agents should NOT do). Keep it concise.
```

Claude Code reads your *actual code* and generates an AGENTS.md specific to your product. **Review the boundaries section carefully.** This is where you tell agents what

they should never do. Commit it.

CLI

```
> Build a CLI wrapping our top 5 API endpoints. Python with typer
and httpx. JSON output by default. Error codes in error messages.
Auth via PRODUCT_API_KEY env var. Thorough help text. Test locally.
```

Run a few commands. **The test: does the `--help` output make sense on its own?** If the agent reads that to decide how to use the tool. If it confuses you, it confuses the agent.

MCP Server

```
> Build an MCP server using Python and FastMCP. Read-only tools for
our top 5 use cases. Each tool description should explain what it
does and when to use it, written as if onboarding a new team member.
Input schemas with types. Auth via env var. Read-only only.
```

Claude Code generates the server. **Now comes the part where you earn your keep as a PM.**

Read every tool description. If a description says “manages tasks,” rewrite it to “manages tasks for a project, filtered by status. Use this when checking what work is pending.” Stripe and Linear got this right. Your descriptions should be just as clear.

Test and Ship

```
> Add our MCP server to your config. List all tools. Do a read
operation. Trigger an error. Chain two operations. Tell me
what felt confusing.
```

This is where you see your product through an agent's eyes. If Claude Code picks the wrong tool or gets confused, every other agent will too. Fix descriptions until it's intuitive.

Ship it:

```
> README with one-command setup for Claude Code, Cursor, VS Code.  
Example prompts. Create a PR with everything.
```

Full agent access stack. Deployed from a single session.

Already Have an MCP Server? Diagnose It.

If your team already shipped an MCP server, the question is no longer “do we have one?” It’s “is it actually good?” Most aren’t. Here’s how to find out.

Diagnostic 1: Is your MCP server getting picked?

If agents have access to multiple tools (yours and a competitor’s), the one with better descriptions wins. Connect Claude Code to your MCP server and a competitor’s. Ask it to complete the same task. Which tools does it reach for first? If it picks the competitor’s tool, your descriptions are too vague.

Diagnostic 2: Description quality audit.

```
> Connect to our MCP server at [URL]. List all tools.  
For each tool, evaluate:  
1. Is the description specific enough to pick the right tool on first try?  
2. Are input schemas complete with types and descriptions?  
3. Are readOnlyHint/destructiveHint annotations set correctly?  
4. Are error messages informative enough for an agent to self-correct?  
  
Rank issues by impact on agent usability. Suggest rewrites  
for the worst descriptions.
```

Most MCP servers ship with vague descriptions because engineers wrote them. The PM pass is where descriptions go from “manages tasks” to “Get all tasks for a pr

filtered by status. Use when checking pending work.” *That specificity gap is the difference between being picked and being skipped.*

Diagnostic 3: Tool coverage vs. API drift.

```
> Compare our MCP server's tool coverage against our API.  
Which API endpoints don't have corresponding MCP tools?  
Which ones should, based on frequency and agent use cases?  
Flag any tools that are out of sync with the current API.
```

MCP servers rot fast if they're not updated alongside the product. This audit catches drift before your users do.

Diagnostic 4: Is your MCP server driving discovery?

Check whether your MCP server is listed on [PulseMCP](#), Docker's MCP Catalog, the Claude integrations directory. If it's not listed, agents can't find it. Listing is equivalent of submitting your app to the App Store. You also want to track: how unique agent sessions connect per week, which tools get called most, and which never get called (candidates for removal or rewriting).

Prompts for Ongoing Agent Work

These are the prompts you'll reuse every sprint. Bookmark them.

Competitive analysis:

```
I'm a PM at [company] building [product]. Competitors:  
1. [Competitor 1]  
2. [Competitor 2]  
3. [Competitor 3]  
  
Research each: CLI? MCP server? OpenAPI docs? AGENTS.md?  
Listed in MCP registries? Frame as competitive gap analysis  
for a strategy meeting.
```

If a competitor has an MCP server and you don't, every Claude Code user, every Cursor session picks them by default. That's the framing that gets executive attention.

Scoping what to expose:

Our product's core features and API coverage:

[List features with: full API, partial API, no API]

Recommend 5 features to expose to agents first. Prioritize by frequency, complexity (single tool call?), risk, and API readiness.

For each, draft the MCP tool name and 2-sentence description.

The output becomes your PRD scope. It also surfaces API coverage gaps, which often the real bottleneck nobody wants to talk about.

Writing MCP tool definitions:

These are the most important piece of copy you'll write for the agent channel. Think of it as the landing page copy for agents. It's what landing page copy is to humans.

Top [X] API endpoints for [product]:

[Paste API docs]

Generate MCP tool definitions: name (snake_case), description (what it does + when to use it), input schema, output schema, annotations (readOnlyHint, destructiveHint).

Write descriptions the way Stripe and Linear do: specific enough that an agent knows which tool to pick on first encounter.

Review the output obsessively. Vague descriptions = agents skip you for a competitor.

(For broader PM prompts, my [PM Prompt Library](#) and [Context Engineering](#) guide cover full stack.)

7. Where Teams Go Wrong

I've seen enough MCP launches to spot the patterns that kill agent adoption. Here are the seven most common, ranked by how often I see them.

The 7 Agent Access Mistakes

Aakash C

1 CLI Checkbox

Ship 3 endpoints, human-readable output, call it done

JSON output by default. Human-readable is the optional flag.

2 Vague Descriptions

Manages tasks' tells agents nothing about when to use it

Write as if onboarding a new team member.

3 Write Access Day One

Full CRUD, no guardrails, agent deletes something in prod

Start read-only. Add write with destructive Hint annotations.

4 Browser Auth

Auth flow requires a browser. Breaks headless agents

OAuth 2.0 device flow. Test from a headless environment.

5 Building in Isolation

MCP server drifts from product in 3 months.

Same codebase, same CI pipeline, same PR.

6 Ship, Don't List

MCP server exists but isn't on any registry.

Submit to PulseMCP, Docker Catalog, Claude directory.

7 No Agent Metrics

No tracking of agent calls, error rates, or tool usage

Instrument agent-originated API calls as a separate segment.

The fix for all 7: Treat the agent as a first-class user with a PM who owns the experience.

Mistake 1: The CLI Checkbox

The team ships a CLI that wraps three endpoints, outputs human-readable table instead of JSON, and calls it done.

This is like launching a “mobile app” that’s just your desktop site in a webview. It technically exists. Nobody uses it. Agents need structured JSON, parseable error environment variable auth.

A CLI is a product that needs the same care as your GUI. Every command should output JSON by default. Add `--human` or `--pretty` for readable output. *The default consumer is now a machine.*

Mistake 2: The Vague Description

An MCP tool that says “Manages tasks” tells an agent nothing. When to use it? What does it need? What comes back?

An agent encountering that will skip it for a competitor’s tool that says “Create a task in a project. Use when adding a work item. Requires project_id and title.” The specificity gap between good and bad [tool descriptions](#) is the gap between getting picked and getting ignored.

Research backs this: at 30+ tools with overlapping descriptions, agents start selecting the wrong one. At 100+, failure is **nearly guaranteed**.

This is the single most common failure I see. And the fix is the simplest: write every tool description as if you’re onboarding a new team member who can’t ask follow-up questions. What does it do? When would they reach for it? What does it need from them?

Stripe writes “review payments, troubleshoot declines, process refunds.” Match that bar.

Mistake 3: Write Access on Day One

Full CRUD through the MCP server, no guardrails. An agent does something destructive. Everyone panics. Agent access gets rolled back *entirely*.

Linear, Stripe, Sentry all started read-only and layered in write access with appropriate workflows. Follow them. Phase 1 is read-only. Phase 2 adds write with destructiveHint annotations and confirmation flows.

That's the right sequence. Skip it and you'll spend a quarter rebuilding trust with security team.

Mistake 4: Browser-Dependent Auth

This one kills adoption silently. Your MCP server works great in demos because someone already has a session cookie. Then an agent tries to connect at 3am and login page it can't navigate.

OAuth 2.0 device flow solves this: show a URL and code in the terminal, user app in browser, agent gets the token. API keys for server-to-server.

The test is simple: try authenticating from a headless environment with no browser window. If it doesn't work, agents can't use you.

Mistake 5: Building in Isolation

Team ships an MCP server, nobody maintains it, tools drift out of sync within months. This happens every time agent access lives in a separate repo maintained by "that one engineer who's into AI."

The rule: new GUI capability = new MCP tool. Same sprint. Same PR. Same CI pipeline. Same review process.

Agent access that isn't coupled to the product codebase rots within a quarter. Make it part of the [definition of done](#), or don't bother building it.

Mistake 6: Shipping and Not Listing

You built a great MCP server but didn't list it anywhere.

Submit to PulseMCP, Docker's MCP Catalog, and the Claude integrations directory the week you ship. Put the URL in your API docs, README, and developer landing page.

This is the equivalent of building an iOS app and never submitting it to the [App Store](#). If agents can't find you, *you don't exist*.

Mistake 7: No Agent Metrics

You shipped the MCP server but have no idea if anyone's using it. No tracking of agent-originated calls, no tool-level usage data, no error rate monitoring.

You wouldn't launch a feature without [analytics](#). Same applies here. Segment agent-originated API calls in your existing analytics. Track calls per tool, error rates, and **which tools never get called**.

That last category matters most. Tools that never get called are either poorly designed or shouldn't exist. Both are problems a PM can fix in an afternoon.

What's Next

The teams getting this right share one thing. They treat the agent as a first-class citizen with a PM who owns that experience. The PM reviews tool descriptions the way they review [UI copy](#). Tests the MCP server the way they'd test [onboarding](#). Tracks agent metrics the way they'd track DAU.

That's the job now.

If you do nothing else after reading this, do three things this week:

1. Run the 5-question audit
2. Pick your top 5 agent use cases
3. Ship AGENTS.md before Friday

That alone will put you ahead of most teams still treating agent access like a 2000 problem.

And keep this guide bookmarked. This is one of those shifts where the winners v
revisit the playbook every quarter, not once.

POLL

What did you think of today's post?

Awesome - 5/5

Okay - 3/5

Bad - 1/5

5 resources to go deeper:

1. [MCP explainer and live demo with Zapier CTO](#)
2. [AI PM upskilling with Pendo CEO \(MCP in action\)](#)
3. [Context engineering.guide](#)
4. [How to build AI products](#)
5. [Claude Code guide](#)



Recommend Product Growth to your readers

Deep dives into succeeding as a PM, product leadership, and how to get your next PM job.

Recommend



93 Likes · 7 Restacks

[← Previous](#)[Next →](#)

Discussion about this post

[Comments](#) [Restacks](#)

Write a comment...



Billy Webb  Mar 7

♥ Liked by Aakash Gupta

This is literally the most timely piece. I have an offsite about this subject next week.

♥ LIKE (1) 💬 REPLY

1 reply by Aakash Gupta

1 more comment...

© 2026 Aakash Gupta · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture